OPEN UNIVERSITY OF MAURITIUS

a.

Academia

A cute React Native stationery shop app

ASSIGNMENT

BSc (Hons) Computer Science – Year 2 Semester 2

Module: Mobile Technologies – OUBs033222

Tutor: Mr Yudish Badal

Student name	[Enter your name]
Student ID	[Enter your student ID]
Submission deadline	23 November 2026, 23:55

Prepared with reference to the assignment brief supplied for Mobile Technologies.

Overview

This report addresses both questions in the Mobile Technologies assignment. Question 1 discusses five ways artificial intelligence can be integrated into mobile applications and explains the value each can provide. Question 2 presents Academia, a fictional stationery-shop app designed and implemented in React Native with a cute pastel visual identity and five required screens.

Academia is a front-end prototype for students who want to browse stationery, find products quickly and organise a shopping bag through a friendly, easy-to-use mobile interface. The included source code, local product artwork, screen previews and setup guide accompany this report.

Question 1 — Five examples of AI in mobile applications (10 marks)

Artificial intelligence (AI) can help a mobile app recognise patterns, interpret language and images, predict user needs, and personalise content. The examples below describe practical integrations and the ways they can improve functionality or user experience.

1. Personalised recommendations

A shopping, music or video app can use AI-driven recommendation algorithms to rank products, playlists or content based on a user's past interactions, stated preferences and patterns shared by similar users. The app can show more relevant items on its home screen, reduce the time spent searching, and help people discover products they might otherwise miss. Spotify explains that many recommendations are personalised to each listener, although human editorial choices may also contribute (Spotify, 2026). Developers should provide privacy controls and avoid trapping users in a narrow set of recommendations.

2. Camera-based translation and text recognition

A mobile app can combine optical character recognition (OCR) with AI translation so a user can point a phone camera at printed text and view a translation. It can be useful for signs, labels, instructions, worksheets or menus. This makes information more accessible to people who do not know the original language and reduces the need to type unfamiliar text manually. Google Translate's instant camera feature uses neural machine translation; Google also describes downloaded language support for offline use (Gu, 2019). Results can still be imperfect when text is blurred, curved or handwritten.

3. Image recognition and visual search

An app can use a phone camera or photo upload to identify objects and suggest matching categories or related products. For example, a stationery shop could let a customer photograph a pen or notebook and display similar items. Google ML Kit provides image labelling and object-related vision capabilities that developers can integrate into mobile apps (Google Developers, n.d.-a). This can make discovery faster, support users who do not know a product name, and create more interactive shopping experiences. The app should indicate when recognition is uncertain and let users correct the result.

4. Predictive typing and smart replies

AI can predict a likely next word, correct a typo, or suggest a short response in a messaging or productivity app. Microsoft SwiftKey describes learning a person's writing style to suggest the next word, while ML Kit includes a Smart Reply API for generating reply suggestions (Microsoft, n.d.; Google Developers, n.d.-b). These features reduce typing effort, speed up common tasks and can assist users who find lengthy typing difficult. Suggestions should remain optional because a prediction may misunderstand the user's meaning or tone.

5. An on-device AI study assistant

A learning or productivity app could integrate a suitable machine-learning model to help classify lecture notes, summarise selected material, generate revision questions or answer common questions. When a suitable model runs on the device, it may respond without a continuous network connection and can reduce the amount of personal content sent to a server. Apple's Core ML documentation explains that models can make predictions in an app and can be run or adapted on-device (Apple, n.d.). The developer must choose an appropriate model, protect user content, test the quality of generated answers and tell users that AI output may contain errors.

Question 2 — Academia stationery shop app (20 marks)

App overview and design

Academia is a fictional stationery retailer aimed at students. The interface uses blush pink, lavender, cream, pale blue and mint, rounded cards, short friendly labels and custom local product illustrations. The visual language is intended to feel playful without making prices, controls or navigation difficult to find.

The project is built with React Native and Expo. React Native core components are used to create the interface; React hooks manage the screen state, search terms, filters, favourites, profile information and shopping cart. Expo is used to run and test the project on a mobile device (Expo, n.d.). The current project manifest targets Expo SDK 57 with React Native 0.86.

Functional features

Screen	User interaction and functionality
Login / Sign-up	Switch between login and registration; validate a basic email format and password length; continue as a guest.
Home	Browse ten stationery products, search by name/category/brand, select a category, mark favourites and add products to the bag.
Product detail	View the selected product illustration, name, brand, category, rating, description and price; add it to the bag.

Screen	User interaction and functionality
Cart	Increase or decrease quantities, remove items, calculate line totals/subtotal/delivery/total, and show a checkout preview.
User profile	View name and email, edit and save details, view simple account statistics and log out.

Navigation and implementation choices

- Bottom navigation provides quick access to Home, My bag and Profile. The product detail page has a back control to return to browsing.
- A search field matches text against product names, categories and brands. The Refine sheet combines category, brand and price-range filters.
- The cart recalculates amounts from the selected quantities. For this prototype, delivery is Rs 40 when the subtotal is below Rs 500 and free at Rs 500 or above.
- React Native Animated provides a short screen fade, while pressed-state feedback, a confirmation toast, favourite toggles and quantity buttons provide interaction.
- Product illustrations are stored locally, so catalogue artwork does not rely on a third-party image URL or require a backend connection.

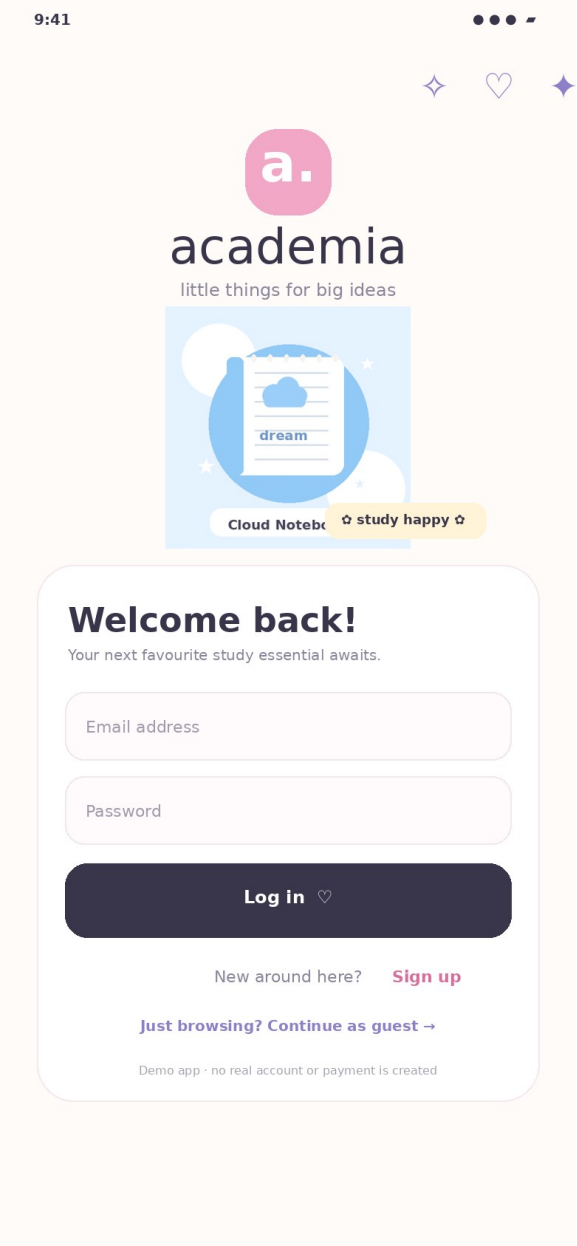
Prototype boundary

This submission implements the required front-end flow, not a production retail service. Login and sign-up are demonstration-only checks; there is no account server, database, inventory synchronisation, persistent storage or payment gateway. Cart and profile data are held in app memory and reset when the app restarts. The checkout control therefore displays a preview message rather than charging a payment method.

Screen design previews

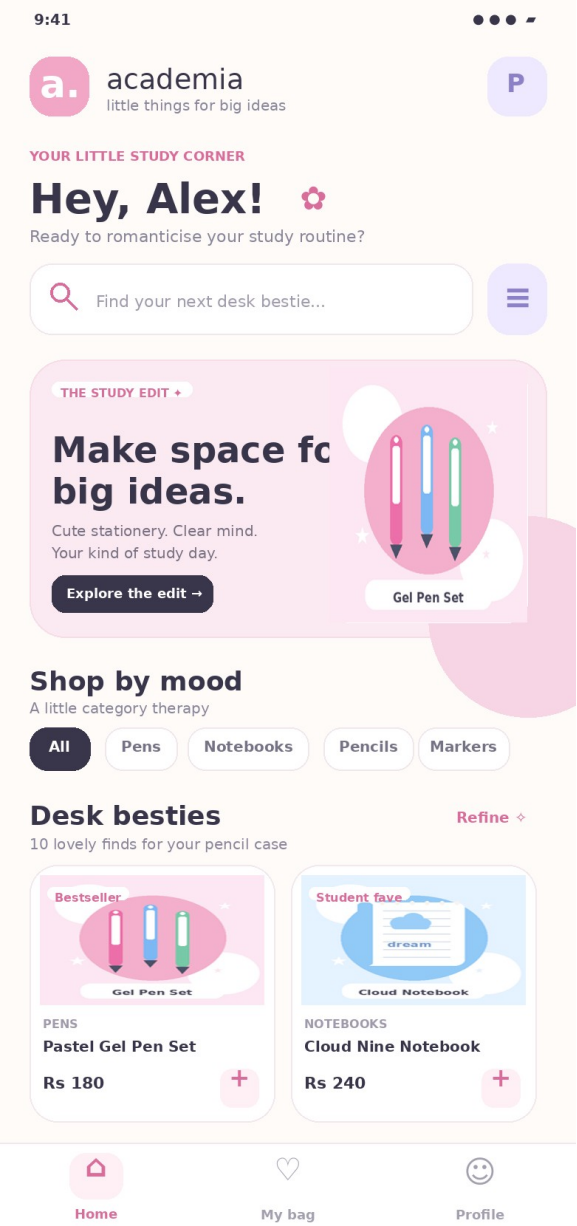
The five illustrations below show the intended layout of the corresponding React Native screens. They are design previews rendered from the Academia UI specification and local product artwork, not captures from a running Expo session. For final submission, capture new screenshots from the app running on Expo Go or an emulator if the tutor requires live application evidence.

Figure 1. Login / Sign-up



Users can log in, switch to account creation or browse as a guest. Email format and basic password-length checks provide immediate feedback.

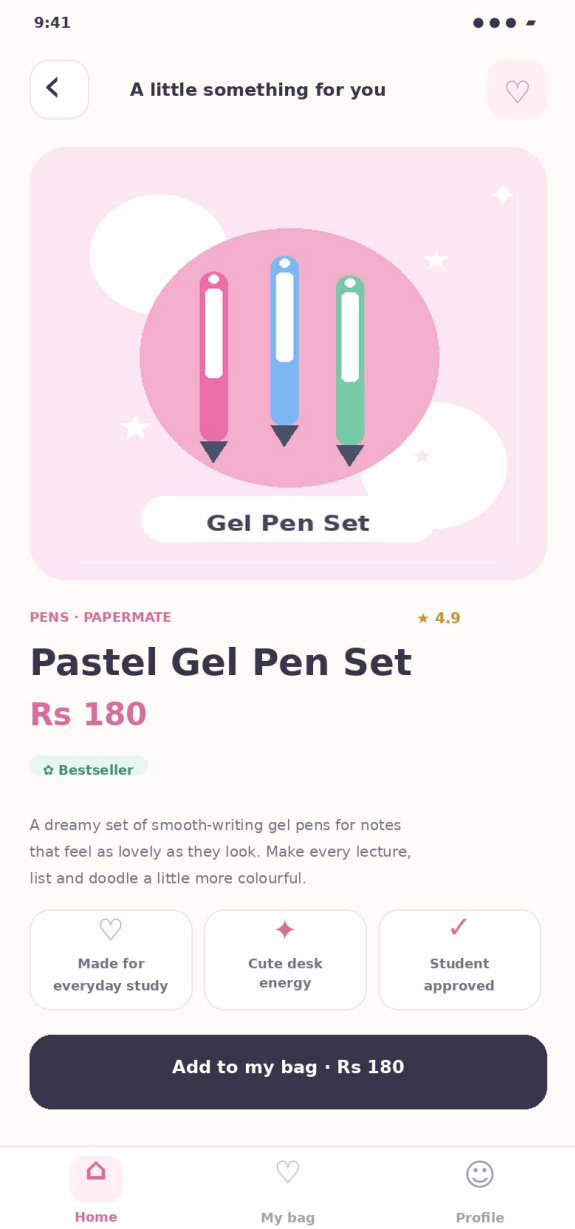
Figure 2. Home and catalogue



The home screen includes the Academia header, search, category shortcuts, a promotional hero card and two-column product cards. Refine opens category, brand and price filters.

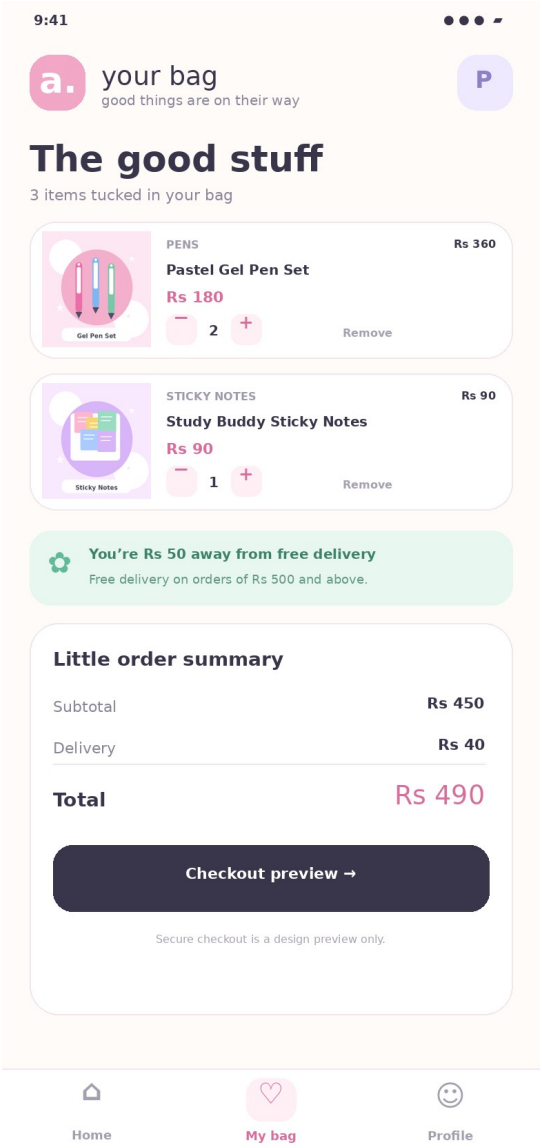
Product and shopping screens

Figure 3. Product detail



The selected product page presents the illustration, category, brand, rating, name, price and description. The heart control saves or removes a favourite; the primary button adds the product to the shopping bag.

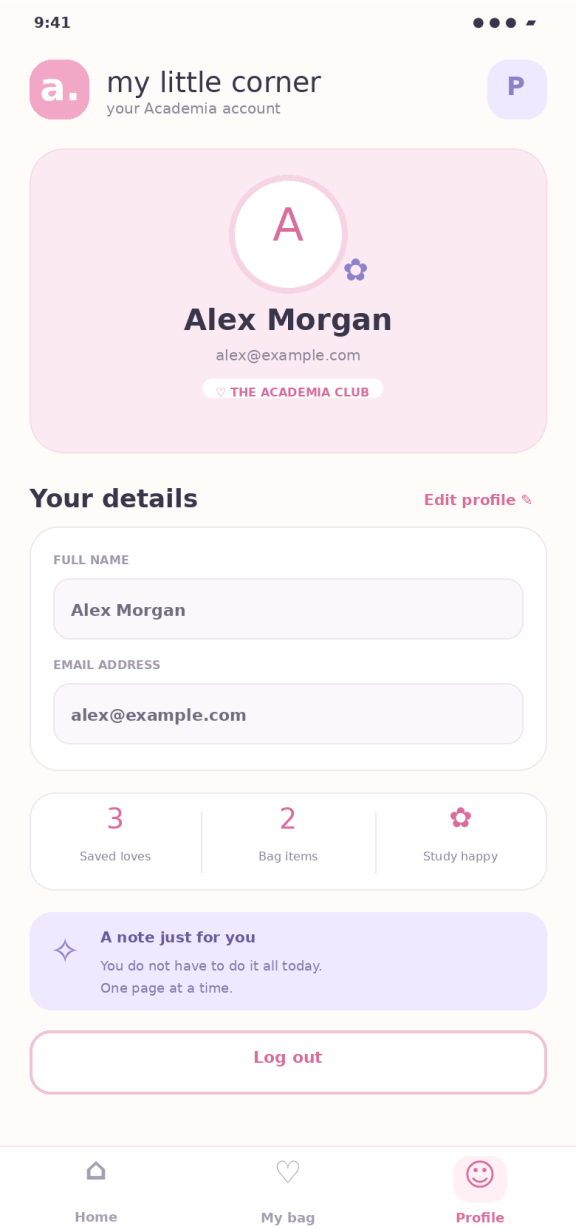
Figure 4. Cart



The cart displays each item's quantity and line total. Quantity changes and removals update the totals. The summary shows subtotal, delivery and total in Mauritian rupees; checkout is clearly labelled as a preview.

Profile screen

Figure 5. User profile



The Profile screen displays the current name and email. Selecting Edit profile enables the inputs and reveals a save action. The user can also view simple bag/favourites counts and log out.

Testing checklist

The following manual tests should be run when the project is opened in Expo Go. The expected outcomes describe the behaviour implemented in the source code; they are not a claim that a live runtime test was completed in the document-generation environment.

Test	Action	Expected result
Login validation	Enter an invalid email or a password shorter than four characters.	A validation alert appears; the app stays on the login screen.
Guest access	Tap “Continue as guest”.	The app opens the Home screen without account credentials.
Search	Search for “notebook”.	The catalogue filters to matching stationery products.
Category and filters	Choose Pens, then filter by brand or price range.	Only products matching all active filters remain visible.
Product detail	Open a product card and tap Add to my bag.	The detail screen shows product information and an add-to-bag toast appears.
Cart calculation	Increase a quantity, decrease it, and remove an item.	Line totals, subtotal, delivery and total recalculate.
Delivery threshold	Make the subtotal lower than Rs 500, then equal to or above Rs 500.	Delivery is Rs 40 below the threshold and free at or above it.
Profile and logout	Edit name/email, save, then log out.	Saved values update in the current session and logout returns to Login.

Short screen-recording plan

A screen recording of around 2–3 minutes can demonstrate the app without slides. Suggested sequence:

0:00–0:20 Introduce Academia, show Login / Sign-up and continue as a guest.

0:20–0:55 On Home, search the catalogue, switch a category and use Refine for brand and price filters.

0:55–1:25 Open a product, show its details, tap the favourite control and add it to the bag.

1:25–2:00 In My bag, change quantities, remove an item, explain the totals and press checkout preview.

2:00–2:30 Edit the profile, save the details and log out; mention that accounts and payment are demo-only.

Conclusion

Academia meets the five-screen brief with search, filters, favourites, cart calculations and profile editing in a consistent pastel design. A production version would need secure authentication, persistent storage, a product database and real payment integration.

References (Harvard style)

Apple (n.d.) Core ML. Apple Developer Documentation. Available at: https://developer.apple.com/documentation/coreml (Accessed: 16 October 2026).

Expo (n.d.) Create a project. Expo Documentation. Available at: https://docs.expo.dev/get-started/create-a-project/ (Accessed: 16 October 2026).

Google (2019) Google Translate's instant camera translation gets an upgrade. The Keyword, 10 July. Available at: https://blog.google/products-and-platforms/products/translate/google-translates-instant-camera-translation-gets-upgrade/ (Accessed: 16 October 2026).

Google Developers (n.d.-a) ML Kit: Image labelling for Android. Google for Developers. Available at: https://developers.google.com/ml-kit/vision/image-labeling/android (Accessed: 16 October 2026).

Google Developers (n.d.-b) Generate smart replies with ML Kit on Android. Google for Developers. Available at: https://developers.google.com/ml-kit/language/smart-reply/android (Accessed: 16 October 2026).

Microsoft (n.d.) Microsoft SwiftKey. Available at: https://www.microsoft.com/en-us/swiftkey (Accessed: 16 October 2026).

Spotify (2026) Understanding recommendations on Spotify. Spotify Safety and Privacy Centre, updated 12 March. Available at: https://www.spotify.com/uk/safetyandprivacy/understanding-recommendations/plain (Accessed: 16 October 2026).

Submission reminder

Before uploading to Blackboard, replace the two cover-page placeholders, open the app from the ZIP and run the manual tests, capture real running-app screenshots if required, and record the short walkthrough video. The assignment brief requests the report/document, short screen recording and complete source code in ZIP format.